

A Matheuristic for the Integrated Healthcare Timetabling Problem (IHTC-2024)

Ahmed Elhefnawy Weiyi Shen Qiaoyan Zeng

Advanced Modeling and Optimization, Group Project
Technical University of Munich, Summer Term 2026

The complete implementation is available at
<https://github.com/ahmedelhefnawy21/ihtp-matheuristic-for-the-IHTC-2024-timetabling-problem>

Contents

1	Problem Description	1
2	Solution Approach	1
3	Implementation	6
4	Numerical Results	7
5	Discussion and Conclusion	8
A	The operating-theatre and nurse models	13
B	Seed distribution (five fixed seeds)	13
C	Component-level cost breakdown	14
D	Certified lower bounds	15
E	The nurse-aware admission audit	16
F	A nurse-aware upper layer: a pre-registered test with a placebo control	17
G	Where the nurse gap sits: a structure comparison and a budget probe	19
H	Held-out evaluation on the hidden instances	20

1 Problem Description

The decision problem, in our own words. A hospital plans a timetable over a horizon of D days ($D \in \{14, 21, 28\}$ in the public instances). Each day has three shifts: early, late, and night. The input describes patients waiting for surgery, occupants already in the hospital (with fixed rooms and stays), rooms, nurses, surgeons, and operating theatres (OTs). For each admitted patient, the hospital chooses an admission day, a room that is kept for the whole stay, and an OT for the surgery. The surgery takes place on the admission day. For every room that holds a patient or an occupant in a given shift, the hospital assigns exactly one on-duty nurse. The competition rules of Ceschia et al. (2025a) define the IHTP. It joins three classic subproblems (Ceschia and Schaerf, 2011; Cardoen et al., 2010; Burke et al., 2004) that share the same beds, staff, and theatres. These are patient admission scheduling (PAS), surgical case planning (SCP), and nurse-to-room assignment (NRA). Some patients are mandatory and must be admitted within a time window. Others are optional and may be postponed. A plan is feasible when it meets the eight hard constraints H1 to H8. Among feasible plans, the goal is to minimize a weighted sum of the eight soft costs S1 to S8. Section 2 lists both.

Sets, parameters, and decision variables. The tables below list the notation. The decision variables are binary, the standard way to write an assignment model (Wolsey, 1998). We keep the room and the admission day in one variable (x) because a room is held for the whole stay. The theatre is kept in a separate variable (y) because it is used only on the admission day.

Sets

$\mathcal{D} = \{0, \dots, D-1\}, \mathcal{S}$	days ($D \in \{14, 21, 28\}$) and the $3D$ shifts; shift s is on day $\lfloor s/3 \rfloor$
$\mathcal{P} = \mathcal{M} \cup \mathcal{P}^O, \mathcal{O}$	patients (mandatory $\mathcal{M}$, optional $\mathcal{P}^O$) and occupants
$\mathcal{R}, \mathcal{N}, \mathcal{U}, \mathcal{T}$	rooms, nurses, surgeons, theatres; $\mathcal{N}_s \subseteq \mathcal{N}$ on duty in shift s
$\mathcal{A}$	the ordered list of age groups
$\mathcal{P}^A, \mathcal{P}^B$	patients of gender A and gender B (a partition of $\mathcal{P}$)
$\mathcal{R}_p, \mathcal{D}_p$	compatible rooms $\mathcal{R}_p \subseteq \mathcal{R}$ and legal admission days $\mathcal{D}_p = [\rho_p, \lambda_p]$ of p

Parameters

cap_r	beds in room r
$A_{t,d}, T_{u,d}^{\text{surg}}$	daily minute budgets of theatre t and surgeon u (0 = unavailable)
$\sigma_n, L_{n,s}$	nurse skill level and shift maximum load
$\rho_p, \lambda_p, \ell_p$	release day, last admissible day (= due day if mandatory, $D-1$ otherwise), length of stay
$\delta_p, u_p, g_p, \text{age}_p$	surgery duration, surgeon, gender, age-group index of p
$w_{p,j}, \text{req}_{p,j}$	workload and required skill of p at stay-shift $j = 0, \dots, 3\ell_p-1$
$\text{occ}_{r,d}^A, \text{occ}_{r,d}^B$	occupants of gender A / gender B present in room r on day d
$W_1, \dots, W_8$	the eight soft-constraint weights

Decision variables (all binary)

$x_{p,d,r}$	patient p admitted on day d into room r (only for $d \in \mathcal{D}_p, r \in \mathcal{R}_p$)
z_p	optional patient p left unscheduled
$y_{p,d,t}$	surgery of p placed in theatre t on day d
$v_{n,r,s}$	nurse n covers room r in shift s (only for $n \in \mathcal{N}_s$)
$g_{r,d}$	gender indicator of room r on day d (1 = gender B; PAS-MIP auxiliary)

2 Solution Approach

Decomposition: how and why we split the problem. A single MIP over all four variable families, including the nurse variables, is too large to solve within our budgets. Even the admission-only subset exhausts its work limit on the largest instances. We count the variables of Section 1

from the instance sizes. The binaries alone come to about 4×10^4 on a median instance and 4×10^5 on the largest. We therefore split the problem into an upper layer and a lower layer. This is the standard decomposition for the IHTP, and Ceschia et al. (2025b) report it among the strong entries to the competition. These entries differ in which part leads the search. The method of Garcia Tercero et al. (2026) is led by local-search neighborhoods. Ours is led by the exact admission MIP, because the admission terms dominate the objective and an exact model packs patients in well (Section 2). The upper layer sets the admission day, room, and theatre (variables x, y, z). This fixes all hard constraints except nurse coverage, and it fixes the patient-side soft costs S1, S5, S6, S7, S8. The lower layer assigns nurses (variable v), which fixes the coverage constraint H8 and the nurse soft costs S2, S3, S4. We solve the two layers with the following pipeline. The pipeline pairs a heuristic search with exact mixed-integer programs, which is the matheuristic approach surveyed by Maniezzo et al. (2010). Each stage is present because the ablation in Section 5 shows that adding it lowers the overall cost. Anything that did not help was dropped. No component is included just to look advanced. Each is kept only if it brings a measured improvement, and otherwise it stays out (one candidate that failed this test was not adopted, Appendix F).

$$\text{construct} \rightarrow \underbrace{\text{PAS-MIP}}_{\text{admission}} \rightarrow \underbrace{\text{ALNS}}_{\text{refine}} \rightarrow \underbrace{\text{descent}}_{\text{true cost}} \rightarrow \underbrace{\text{CP-SAT LNS}}_{\text{capacity}} \rightarrow \underbrace{\text{OT-MIP}}_{\text{S5,S6}} \rightarrow \underbrace{\text{NRA-MIP}}_{\text{S2,S3,S4}}.$$

Construction. We quickly build a first plan. Patients are inserted in order: mandatory before optional, then tighter admission window, longer surgery, and longer stay first. The most constrained patients are placed while resources are least committed. Each takes its cheapest feasible day, room, and theatre. An optional patient is admitted only if its cost is below the penalty W_8 for leaving it out. A greedy order can fill a surgeon’s schedule and leave a mandatory patient unplaced. We therefore repair H5 with ejection chains, the compound-move idea of Glover (1996). We move the blocking patients to free capacity, backtracking at most three ejections deep. We also try up to three cheap randomized restarts from slightly varied insertion orders (early stop once one is feasible; best plan kept). On the one instance in the ablation where construction ends infeasible (i16), the PAS-MIP restores feasibility through its mandatory-equality constraints (the scored +PAS cell in Table 2). The ALNS opening feasibility phase remains a backup. It is a ruin-and-recreate repair in the sense of Schrimpf et al. (2000), emptying and rebuilding the congested region around an unplaced mandatory patient. The construction only has to give a usable starting plan: the construct column of Table 2 shows how far it sits from best-known before the optimizing stages run.

PAS-MIP (the admission core). This is the decisive stage, because the objective is dominated by unscheduled optional patients (S8). Admitting more of them is a packing problem that local search cannot solve well, but an MIP can¹. Our admission model is the assignment form of the patient admission scheduling problem studied by Ceschia and Schaerf (2011), matching the code that solves it.² The model uses the admission variable $x_{p,d,r}$ (created only for feasible triples) and a per-room-day gender indicator $g_{r,d}$:

$$\begin{aligned} \min \quad & W_8 \sum_{p \in \mathcal{PO}} z_p + W_7 \sum_{p,r,d} (d - \rho_p) x_{p,d,r} \\ \text{s.t.} \quad & \sum_{r,d} x_{p,d,r} = 1 \ (p \in \mathcal{M}), \quad \sum_{r,d} x_{p,d,r} + z_p = 1 \ (p \in \mathcal{PO}) & (\text{admit; H5, H6}) \\ & \sum_{p \in \mathcal{PA}} \sum_{d' \leq d < d' + \ell_p} x_{p,d',r} + \text{occ}_{r,d}^A \leq \text{cap}_r (1 - g_{r,d}) & (\text{H1, H7, gender A}) \\ & \sum_{p \in \mathcal{PB}} \sum_{d' \leq d < d' + \ell_p} x_{p,d',r} + \text{occ}_{r,d}^B \leq \text{cap}_r g_{r,d} & (\text{H1, H7, gender B}) \\ & \sum_{p: u_p = u} \delta_p \sum_r x_{p,d,r} \leq T_{u,d}^{\text{surg}}, \quad \sum_p \delta_p \sum_r x_{p,d,r} \leq \sum_t A_{t,d} & (\text{H3; H4 in aggregate}) \end{aligned}$$

¹One admission can require many patients to move together (Ceschia and Schaerf, 2011). A MIP weighs those combinations jointly, where local search stalls (Maniezzo et al., 2010).

²Our code has one difference. It computes unscheduled optionals directly as $1 - \sum_{r,d} x_{p,d,r}$ rather than with the explicit variable z_p . The two are equivalent. On instances with more than 300 patients the code also clips each optional patient’s admission-day window to the eight days after release (mandatory windows are untouched). This pruning keeps the model small at little cost, since the delay penalty S7 already favors early admission.

The two gender constraints per room-day force every room-day to hold at most cap_r patients of one gender. Rooms and gender are handled exactly here. Theatre packing (S5, S6) is left to the OT-MIP, and the small-weight age mix (S1) to the room choice in the surrogate and to the descent. This keeps the model small. A greedy pass then completes the plan before the feasibility check that decides adoption: cheapest feasible theatre per surgery, nurses filled greedily. The model bounds H4 only in aggregate, so a patient’s slot can fail a per-theatre check. That rare patient is re-placed in the cheapest feasible slot. In the ablation this stage contributes the single largest drop (Table 2, +PAS column).

ALNS (Adaptive Large Neighborhood Search). Adaptive large neighborhood search (Ropke and Pisinger, 2006) repeatedly removes part of the plan³ and rebuilds it better; its purpose here is to find improvements that no single-patient move can reach, when several patients must shift together. Our five removal operators each target one weakness. *Random* removal diversifies broadly and re-offers a couple of postponed optionals, which repair may admit. *Worst-cost* removal pulls out the most delayed patients (S7). The *empty-a-day* move clears one admission day so its capacity can be re-packed. The *empty-a-theatre* move empties one theatre on one day, trying to close it (S5). The *related* removal of Shaw (1998) extracts the patients most related to a randomly chosen seed (same surgeon, overlapping stay, shared room) so they can be re-packed jointly. Our three repair operators re-insert what was removed, mandatory patients always first. *Greedy* insertion (cheapest feasible slot) is the fast baseline. *Regret-2* insertion places first the patient that would lose the most by waiting (the widest cost gap between its best and second-best feasible placement). *Randomized greedy* samples among each patient’s few near-best slots, so the same removal is not always rebuilt the same way. Operators are drawn with probabilities adapted to their recent success, the adaptive scheme that gives the method its name. Each draw earns a graded reward, largest for a new best plan. Every 100 iterations the weights are smoothed toward each operator’s average reward. Moves are accepted by the simulated-annealing rule of Kirkpatrick et al. (1983): a worse plan is accepted with a probability that shrinks as the search cools. This lets the search escape local optima early and settle later. The starting temperature comes from thirty probe moves on the instance itself, set so we accept about forty percent of worsening moves at the start; it then decays geometrically for the rest of the run. Feasibility is maintained by construction, so the only violation the search can create is an unadmitted mandatory patient, which we handle with a large penalty. At periodic checkpoints (every 1,500 iterations) the search also runs a deterministic admission sweep: every postponed optional whose best slot costs less than W_8 is inserted. For each one with no feasible slot, a bounded ejection chain is tried. It is kept only if the surrogate cost drops. It gives the search a cheap version of the packing the exact models do. The ALNS has two measured roles. It reduces the secondary costs the admission model does not optimize. Its sweep recovers admissions where the time-limited PAS-MIP left optionals unadmitted. In the +ALNS column of Table 2, the large i27 drop is consistent with recovered admissions (i27 prices each admission at $W_8 = 500$, Table 4). The column is neutral where its targets are already minimal.

Descent (true-cost local search). The ALNS loop scores the fast surrogate, so a short descent follows whose accept test is the full objective. For each admitted patient it tries every feasible day, including its current one, each with the surrogate-cheapest room and theatre for that day; only the exact current placement is skipped. Only one candidate per day is tried, since each candidate requires a complete rescore, with the nurses re-assigned, and the room and theatre choices interact only through the day. The descent takes the first relocation that improves and keeps a move only when the complete, nurse-staffed score strictly drops. Its purpose is to reduce the nurse terms the surrogate does not price, plus the age mix (S1) left over; its contribution is the +descent column of Table 2.

CP-SAT LNS (the capacity-creating step). The single PAS-MIP times out on the largest

³The random, worst-cost, and related removals draw their size uniformly between one and one seventh of the admitted patients, at most 25. That size is large enough for joint moves and cheap to repair. The day and theatre removals clear their whole target. Partial clearing would leave the packing in place.

instances (the drop from the +descent to the +LNS column of Table 2 shows what the timeout leaves unrecovered). No small move can re-pack a full schedule to fit one more optional patient. We therefore free every still-unscheduled optional patient, plus everyone admitted inside a random window of days, and re-pack that window with an exact model. The freed set is capped near 110 patients, and the window width scales with instance size. When the set still overflows, admitted optionals are randomly subsampled, never the unscheduled or mandatory patients. This is a large neighborhood search, the destroy-and-rebuild scheme surveyed by Pisinger and Ropke (2010), with an exact re-pack in place of a heuristic one. The frozen patients’ resource use is subtracted as residual capacity, so the small model stays valid. We solve it with CP-SAT, the constraint solver in OR-Tools. Its reified constraints (constraints switched on and off by a boolean variable) model the gender, open-theatre, and surgeon-transfer indicators directly, and its deterministic time budget keeps the many small repeated solves reproducible. Admissions raise nurse cost, so each round bundles several re-packs, screened on the costs the re-pack model itself counts (S5–S8). The nurse re-solve then runs once per round rather than once per re-pack. A round is kept only after the nurses are re-optimized and only if the full objective strictly improves, so the stage is monotone. We run four rounds, since profiling found little gain past four (the budget note in `ihnp/experiments.py`). The cap keeps each re-pack model near one size, so the fixed per-solve budget fits every round. In our experiments this stage lowers the cost in every row of Table 2 (+LNS column); it is where the last hard-to-place optional patients are admitted.

OT-MIP. Operating-theatre scheduling is its own research field, reviewed by Cardoen et al. (2010). With days and rooms fixed, each day is a small independent assignment problem. We pick the theatre $y_{p,d,t}$ for each surgery to minimize open theatres (S5) and surgeon transfers (S6) under H4. Each day has a few surgeries, so each of these models is tiny and runs under a fixed deterministic per-day work budget. The stage’s small remaining gain is the +OT column of Table 2.

NRA-MIP. Nurse-to-room assignment is the staffing side of the plan, closest to the nurse rostering problems surveyed by Burke et al. (2004). The roster here is fixed input, and the task is to assign on-duty nurses to rooms. With the layout fixed, the nurse variable $v_{n,r,s}$ places one on-duty nurse per occupied room-shift (H8). Overflow variables $o_{n,s} \geq 0$ model excess workload (S4), and the link $c_{e,n} \geq v_{n,r_e,s}$ counts distinct nurses per patient, so $S3 = \sum_{e,n} c_{e,n}$. The objective $W_2S2 + W_3S3 + W_4S4$ is minimized over the whole horizon, since continuity (S3) couples every shift of a patient’s stay; no shift-by-shift solve could trade this off. The greedy nurse assignment is a feasible warm start: a chronological sweep, so a patient’s earlier nurse can be picked again for continuity. It staffs the rooms with the highest required skill first, while the skilled nurses are still lightly loaded, and picks each nurse by marginal cost under the true weights. The sweep, however, fixes each shift in turn and cannot look ahead, so this exact solve recovers the skill, continuity, and workload cost the sweep misses (Table 2, +NRA column). Both models appear in full in Appendix A.

How the components interact. Three mechanisms tie the stages into one method. First, *warm starts*: every exact model begins from the current plan. The construction seeds the PAS-MIP, the incumbent theatre placement seeds the OT-MIP, and the greedy nurse assignment seeds the NRA-MIP. The current placements enter the CP-SAT re-pack as hints. Each Gurobi solve therefore starts from a known solution it can only improve in its own objective. The round check described next covers the re-pack. Second, a *keep-best incumbent*: from the descent onward, a stage’s output replaces the incumbent only when the full evaluated objective strictly improves. The capacity-creating LNS and the final exact solves can never make the reported solution worse. The two stages before them, the PAS-MIP and the ALNS, hand their result forward directly (PAS accepted when feasible, ALNS feasible by construction). They re-decide the dominant admission terms rather than refine a fixed plan. Third, a *two-speed objective*: the heuristic’s inner loops and the admission models score a fast upper-layer surrogate that excludes the nurse terms. The incumbent is updated at the ALNS checkpoints, each LNS round, the descent, and the final

Algorithm 1 One run of the pipeline: S is built once (line 1), then improved stage by stage; each SMALL-CAPS call is a stage of Section 2. $z(S)$ is the full objective, all eight soft costs with the nurses re-assigned before scoring (greedily, except at the NRA-MIP calls, which are exact). Lines 2–5 re-decide the admissions and hand their result forward; from line 6 on a change is kept only if it strictly lowers z , so the plan never gets worse. Every stopping rule is an iteration, round, or work budget, never wall-clock, so a fixed seed reproduces the run (same platform and solver versions; Section 4).

Require: an instance and a seed

Ensure: a feasible plan; every plan we report is additionally re-checked by the competition’s official validator

```

1:  $S \leftarrow \text{CONSTRUCT}$   $\triangleright$  a fast greedy plan; hardest-to-place patients inserted first
2:  $P \leftarrow \text{PAS-MIP}(\text{warm start } S)$   $\triangleright$  exact model re-decides every admission, day, and room
3: fill  $P$ ’s theatres and nurses greedily  $\triangleright$  the admission model leaves both to this quick pass
4: if  $P$  is feasible then  $S \leftarrow P$   $\triangleright$  adopted outright: admissions carry most of the objective
5:  $S \leftarrow \text{ALNS}(S)$   $\triangleright$  reshuffles days, rooms, and theatres, with periodic admission sweeps; worse plans accepted early on
6:  $S' \leftarrow \text{DESCENT}(S)$ ; if  $z(S') < z(S)$  then  $S \leftarrow S'$   $\triangleright$  one-patient moves, judged on the full score
7: for 4 rounds do  $\triangleright$  the capacity-creating step; each round bundles several re-packs, each on a fresh random window
8:    $S' \leftarrow \text{REPACKSLICE}(S)$   $\triangleright$  free the not-yet-admitted optionals + a day window, capped near 110 freed; CP-SAT re-solves the slice, the rest stays frozen
9:    $S' \leftarrow \text{NRA-MIP}(S')$ ; if  $z(S') < z(S)$  then  $S \leftarrow S'$   $\triangleright$  exact nurse assignment; keep the round only if cheaper once staffed
10: end for
11:  $S' \leftarrow \text{NRA-MIP}(\text{OT-MIP}(S))$ ; if  $z(S') < z(S)$  then  $S \leftarrow S'$   $\triangleright$  final clean-up: exact theatre assignment, then exact nurse assignment
12: return  $S$ 

```

exact solves. Each update re-scores the full objective with the nurses re-solved, greedily at the checkpoints and in the descent, and exactly thereafter. The surrogate makes many iterations possible, and the full-objective checks keep the reported cost accurate. Algorithm 1 summarizes one full run.

Hard constraints and feasibility. Feasibility rests on two things: the construction and the exact models keep every hard constraint, and the official validator confirms it. H2 (compatible rooms) and H6 (admission window) are held by construction, because we only ever create a variable $x_{p,d,r}$ for a legal day and a compatible room. H1 (no gender mix) and H7 (room capacity) are enforced exactly by the two gender constraints of the PAS-MIP, and are re-checked on every heuristic move. H3 (surgeon time) is enforced by the PAS-MIP and re-checked on every heuristic placement. H4 (theatre time) is enforced by the PAS-MIP only in aggregate (total surgery duration per day within total theatre capacity). The exact per-theatre bound is enforced when theatres are assigned, and checked on every placement and move. The OT-MIP re-optimizes it within capacity, and the official validator confirms it on every final solution. H5 (all mandatory admitted) is secured by the construction, the ejection repair, and the equality constraint in the PAS-MIP. H8 (nurse presence: an on-duty nurse covers every occupied room-shift) is met by the greedy nurse assignment and the NRA-MIP, which chooses $v_{n,r,s}$ only among nurses on duty in that shift. The official validator, our ground truth for feasibility, implements H8 as two checks: **UncoveredRoom** (every occupied room-shift has a nurse) and **NursePresence** (the assigned nurse is on duty in the shift). We never claim a solution feasible on our own evaluator alone. Every reported solution is re-run through the official validator. We report it as feasible only when that run shows zero

violations.

Objective and soft constraints. We re-implemented the official objective in Python and checked it against the validator with a golden test. Our evaluator and the official validator score the same published solutions, and every component must match exactly (Section 3). The search therefore optimizes the objective the competition measures. Each soft cost is reduced at the stage that controls it. Admission (PAS-MIP and CP-SAT LNS) reduces unscheduled patients (S8) and admission delay (S7). The OT-MIP reduces open theatres (S5) and surgeon transfers (S6). The NRA-MIP reduces under-skill (S2), continuity of care (S3), and excess workload (S4). The descent reduces the age mix (S1, the spread between the oldest and youngest age groups sharing a room on a day). The weight W_8 on unscheduled patients is by far the largest (150 to 500 across instances, against at most 50 for the others), so the objective is driven mostly by how many patients are admitted.

3 Implementation

Architecture. The solver is a Python package (`ihtp/`). Its modules are instance parsing, a solution model with incremental cost updates, a full objective evaluator, the constructive and ALNS heuristics, the four exact models, and the experiment scripts. The incremental model keeps running totals of cost and capacity, so testing one move costs time proportional to what the move touches, not to the whole instance. One command (`./run.sh`) builds the validator, runs the tests, solves all instances, computes the certified lower bounds, and writes every result. Every number and table in this report can therefore be rebuilt by running the code on a fresh copy of the repository, with the recorded solver versions and licenses. The appendix experiments have their own scripts under `extensions/`.

Solvers, libraries, and tools. We use **Gurobi 13.0.2** for the admission, theatre, and nurse MIPs and for the lower bound. Its deterministic work-based stopping (`WorkLimit`) also underpins our reproducibility. Gurobi is the only commercial solver we use, under an academic license, as permitted by the assignment. We use the open-source **OR-Tools CP-SAT 9.15** for the capacity-creating search, and **NumPy 2.5** for the data arrays. Feasibility is checked by the official **IHTP_Validator** (IHTC, 2024) (the C++ program from the competition), which we compile from its source.

Instance and solution formats. The program reads the official IHTC-2024 instance JSON files (IHTC, 2024) and writes the solution JSON in the required schema (the admission day, room, and theatre per patient, and the nurse per occupied room-shift). The required results CSV (one line per instance, `i01,<objective>`, or `i01,infeasible`) is written by the same scripts.

Validation. Our Python evaluator guides the search. It matches the official validator component by component (our golden test) on the published i01 reference solution (zero violations, all eight soft costs, and total 3842). It also matches on seven further reference solutions spanning the size range, where we check every hard and soft component. Since i01’s reference solution has zero surgeon transfers and zero excess workload, these seven test the components and edge cases (near-horizon truncation, ~ 500 -patient instances) that i01 cannot. These are feasible published solutions, so this is an agreement test on valid states, not a proof of equivalence for every solution the search visits. It need not be: every solution we report is re-scored by the official validator binary, and we keep the validator’s own objective value as the reported number.

Computational environment. Every run records the machine on its own, in the file `environment.txt`. Each run’s results store the solver versions the objectives depend on, and the

hardware the runtimes depend on. The submitted run used an Apple M5 processor (10 cores), 34.4 GB of memory, and macOS 26.5.2 (arm64), with Python 3.12.13, Gurobi 13.0.2, OR-Tools 9.15.6755, and NumPy 2.5.1. The full run (all 30 instances, five seeds each, across the ten cores) took about 5.8 hours (20,953 seconds) of wall-clock, for the solve and validation. The certified bounds were computed in a separate step, not included in this figure. The per-instance runtime in Table 1 is the solver time summed over the five seeds. The 150 single-threaded runs share the ten cores, so their solver times sum to about 58 hours, roughly ten times the wall-clock. Each solver gets one thread. Several solver threads working together can return a different solution under a fixed budget, which would break fixed-seed reproducibility. We fill the cores by running instances in parallel instead.

4 Numerical Results

Setup, feasibility, and objectives. We ran all 30 public instances. We found a feasible solution for every instance, with zero violations from the official validator, so the feasibility column of Table 1 reads “yes” throughout. We compare each objective with the competition best-known value, which is the row minimum over the published results of the 32 teams reported by IHTC (2024). Apart from the certified gaps of Section 5, every gap in this report measures our objective against this benchmark, so a positive gap means the field has published a better solution. The best-known baseline is a dated snapshot (6 July 2026) stored with the code, so our gaps stay fixed even if the competition site changes later. The metaheuristic is randomized, so we run five fixed seeds per instance and report the best of the five as the headline value, with the seed spread in Appendix B. The four added seeds recover about one point of mean gap.

Results table. Table 1 reports, for each instance, the feasibility, the objective, the best-known value, the gap, and the total runtime over the five seeds. The aggregate gap is 7.8%, total-based: the difference of summed objectives and summed best-known values, divided by the best-known total. Every aggregate gap in this report follows this rule; it weights instances by cost, so cheap instances cannot dilute hard ones. A held-out run on the competition’s later-published hidden instances, made once under a pre-registered protocol after all development ended, reaches 6.10% (Appendix H). We come within 0.5% of the best-known on 2 instances and within 3% on 7. A group of mid-size and hard instances keeps double-digit gaps. The aggregate component analysis below attributes the overall gap mainly to the nurse layer; we do not decompose each instance’s gap separately.

Seed robustness. Across the five seeds the objective is stable on the small instances (a standard deviation of a few tens of units) and spreads on the hard ones (up to about 1300 on i17). The roughly one-point best-of-five recovery stated above is modest, so the quality comes from the method, not from running many restarts. The seeds give a robustness estimate and a reproducible way to get back some of the search variety our deterministic setup gives up.

Reproducibility. The method is deterministic.⁴ It uses no wall-clock stopping rule. The heuristic runs a fixed number of iterations (8,000 in the ALNS). Each solver stops on a fixed amount of work (Gurobi’s WorkLimit, and a deterministic time budget for CP-SAT).⁵ For a fixed seed the solver returns the same solution on the same platform and solver versions, given fixed threads and work budget. Our repeated fixed-seed runs confirm this. One run, on record in Appendix F, matches bit for bit on i01 seed 1. Settings are in `ihhttp/solvers.py`, per-stage

⁴This does not conflict with the randomized metaheuristic: the search makes random choices, but each is drawn from a pseudorandom generator that the run’s seed fixes, so the same seed reproduces the same run.

⁵Both budgets count solver-defined units of work rather than seconds: identical across runs by construction, and the hardware only changes how long one unit takes. Gurobi sizes a work unit near one second of computation.

Table 1: Results on the 30 public instances. Best-known is the benchmark, and the gap is measured against it. Runtime is the total over five seeds, since the headline is the best of those runs and the winning seed’s time alone would understate the compute spent.

instance	feasible	objective	best-known	gap%	runtime (s)	comment
i01	yes	4047	3842	5.34	988.9	
i02	yes	1444	1264	14.24	1182.6	
i03	yes	10605	10490	1.10	124.4	
i04	yes	2021	1884	7.27	6277.6	
i05	yes	12903	12760	1.12	5014.8	
i06	yes	10696	10671	0.23	2434.4	within 0.5% of best-known
i07	yes	5530	5026	10.03	6337.7	
i08	yes	6929	6291	10.14	6671.1	
i09	yes	7150	6682	7.00	5419.2	
i10	yes	21525	20820	3.39	5891.6	
i11	yes	25966	25938	0.11	5490.6	within 0.5% of best-known
i12	yes	13453	12430	8.23	5732.5	
i13	yes	17979	17328	3.76	6317.2	
i14	yes	10011	9746	2.72	6056.8	
i15	yes	13451	12486	7.73	6664.4	
i16	yes	11485	10139	13.28	6461.3	
i17	yes	47970	40535	18.34	7941.1	
i18	yes	37903	37660	0.65	5335.4	
i19	yes	49544	44587	11.12	12845.6	
i20	yes	30098	29098	3.44	6335.2	
i21	yes	28011	24703	13.39	9598.8	
i22	yes	53380	47861	11.53	11116.2	
i23	yes	39230	37550	4.47	12508.9	
i24	yes	34476	33221	3.78	10669.8	
i25	yes	12322	11517	6.99	8935.9	
i26	yes	71048	64613	9.96	8375.0	
i27	yes	62222	51828	20.05	10669.7	worst gap; largest (493 patients)
i28	yes	76176	75172	1.34	6607.5	
i29	yes	14287	12475	14.53	10825.5	
i30	yes	40597	37943	6.99	10072.2	

All 30 feasible. Aggregate gap 7.8%, mean per-instance gap 7.4%. Within 0.5% of best-known on 2; within 3% on 7.

budgets in `ihp/experiments.py`. Only the runtime changes with hardware. The 30 instances are independent, so they run in parallel across all cores without breaking determinism.

5 Discussion and Conclusion

Where the gap comes from and where the computing effort goes. We compared our solutions with the best-known solutions component by component, summed over all 30 instances. The largest cost in the objective is unscheduled patients (S8), but that is not where we lose. A direct count of admitted optionals (not the weighted S8 cost, whose weight varies by instance) shows we admit 2995 against the reference solutions’ 2933, and our weighted S8 is about 2% lower. The gap is therefore not about which optionals we admit. Admission delay (S7) costs us 17% of the gap. Most of the rest comes from the nurse side: continuity (S3) 39%, under-skill (S2)

30%, and workload (S4) 22%, about 90% of the net gap (78% of the gross positive differences). Appendix C and Table 3 give the breakdown and the per-instance counts. We diagnosed two causes, each with a controlled experiment. The nurse-budget sweep ran during development; the notes in `ihttp/experiments.py` summarize it, and its per-instance records were not archived. The registered probe of Appendix G later re-measured the effect with stored records. First, the nurse MIP was given too little solver time. A re-solve with a larger work budget on the same fixed layout cut nurse cost by up to 16% on the hard instances, and by 0% on the small ones that were already optimal. This led to the size-scaled nurse budget the pipeline now uses, 600 to 800 work units rising with rooms times days. The aggregate gap moved from 9.2% at the old flat budget of 120, to 7.9% at a flat 600, to 7.8% with the scaling. It applies to the final nurse solve; the LNS-round re-solves keep a smaller cap, since they only decide round acceptance. Second, a cause remains after the budget fix, and we first read it as purely structural. The admission stage does not see the nurse cost its choices create. A later probe splits it (Appendix G). A larger nurse budget alone closes 6% to 63% of the remaining gap on the largest-gap nurse-heavy instances, and none on a small control. Part of what remains is therefore solver budget, and the rest is structure. These two also carry the computing effort. The admission MIP stops fitting inside its work budget as instances grow, which is why the CP-SAT re-pack stage exists. The sweep showed the nurse MIP starved, which is why that budget is the one we scale with instance size. More seeds help only slightly (Appendix B); the budget notes in `ihttp/experiments.py` record the same for longer budgets, which the probe above corrected for the nurse budget on the largest instances.

The coupling test: a nurse-aware admission audit. The simplest explanation would be that the nurse-blind admission stage admits patients whose staffing cost exceeds the penalty they avoid. A standalone audit tests that reading directly, leaving the pipeline untouched (Appendix E). The pipeline can admit an optional patient, but no stage can later un-admit one against the full objective. The audit adds that missing move. It postpones each admitted optional, one at a time, from our final solutions, and keeps a removal only if the complete objective, nurses included, strictly improves. On i01 and the five largest-gap instances, over 1 061 admitted optionals, it accepts zero removals. The smallest margin is 58 cost units (i27). Every audited admission is individually justified. The damage is not in how many patients we admit. It is in where we put them, meaning which room and which day, and that drives continuity (S3), the largest nurse term. Two fixes follow: pricing an estimate of the placement-side nurse cost into the admission objective, and looping the admission and nurse models rather than running them once. Garcia Tercero et al. (2026), competition finalists, keep the models separate in the same way, with IP-based operators inside a local search (Ceschia et al., 2025b). Appendix F reports a pre-registered test of a static form of the first step, priced into the heuristic upper layer. It passes its declared criteria, but a placebo control⁶ attributes its small aggregate gain to the perturbation itself rather than to the nurse information, which points away from static prices. Appendix G then splits the remaining nurse gap between solver budget and the coupling itself.

Ablation. ⁷ Table 2 adds the stages one at a time; each stage’s measured role is stated with that stage in Section 2. The same standard held beyond the stages: the nurse budget rose only after the controlled experiment above showed it binding, and Appendix F’s priced layer was not adopted after failing its placebo control. The four instances span the size range (i04 to i27), and the stages enter in pipeline order, so neither the subset nor the order is tuned to the outcome.

⁶A placebo control removes the claimed active ingredient, here the real nurse information, and keeps everything else the same; if it does as well, the ingredient did not help (Shadish et al., 2002). A perturbation is any change to the numbers guiding the search.

⁷An ablation is a controlled experiment that measures what one part of a method contributes (Fawcett and Hoos, 2016). We add the stages one at a time, in the fixed order the pipeline runs. The seed and the per-stage work budgets stay fixed, and we record the objective after each. The PAS-MIP and the CP-SAT LNS share the admission terms. Every other stage targets its own cost terms, so one stage’s drop cannot come from giving another

Table 2: Cumulative ablation, adding the stages in pipeline order on four instances that span the size range from the small i04 to the largest i27 (seed 1, so the best-of-5 headline objective can be lower). The PAS-MIP gives the largest drop, from admissions. The CP-SAT LNS and the exact NRA solve lower every row. The ALNS, the descent, and the exact OT solve improve instances where their cost term is not already minimal. Construction alone is infeasible on i16, so it has no scored construct objective there.

instance	construct	+PAS	+ALNS	+descent	+LNS	+OT	+NRA	best-known
i04	2 833	2 505	2 188	2 188	2 022	2 022	2 021	1 884
i13	27 081	18 877	18 877	18 697	18 085	18 055	18 020	17 328
i16	–	12 127	12 127	12 043	11 533	11 533	11 532	10 139
i27	85 614	75 184	68 084	66 464	65 003	64 583	63 896	51 828

Certified optimality gap. The best-known values are upper bounds, not proven optima, so a gap to them cannot say how close we are to the true optimum. We therefore compute a valid lower bound of our own⁸. We solve a relaxation that keeps the admission decision and the two binding resources (surgeon-day and theatre-day time) with the costs they drive, and drops the rest. No dropped constraint or non-negative cost can raise the optimum, so the relaxation’s proven dual bound is a valid lower bound (Wolsey, 1998), the argument Ceschia and Schaefer (2011) use for patient admission scheduling (Appendix D). This gives a certified gap: the true optimum lies between the bound and our objective. The bound is loose because it ignores the nurse and age-mix costs. Both our certified gap (57%) and best-known’s own (46%) are therefore large and track each other across instances. We read the looseness as mostly in the bound, since the pooled best of 32 teams is unlikely to sit 46% above the true optimum.

Strengths and limitations. Every public and hidden run produced a feasible solution, with gaps a few percent above the pooled best of 32 teams. The results rebuild from the repository. Our development and tuning used the 30 public instances, so the headline results are measured on instances that shaped the method. A later pre-registered run on the competition’s hidden instances (Appendix H) adds a held-out measurement: feasible on all 150 runs, at a 6.10% aggregate gap against a smaller benchmark pool. Claims beyond these two instance sets remain untested. The certified lower bound is valid but loose, so we locate each true optimum only within the reported interval. The numbers are tied to the recorded solver versions, budgets, and hardware, and to a Gurobi license, which the exact models and the certified bound require. A different solver version or CPU architecture may change them. We report seed spreads (Appendix B), not formal confidence intervals. The open modeling problem is separate from these limits on the evidence. Appendix F’s static-pricing test left the admission-nurse coupling unresolved. The coupling loop and a larger nurse solve are the future work we propose.

Conclusion. A two-layer matheuristic solves the IHTP feasibly on all 30 public instances and comes within a fraction of a percent of best-known on the closest ones. The aggregate gap is 7.8%, and 6.10% in one held-out run on the hidden instances (Appendix H). We built the method from our own reading of the problem rather than adapting a published solution, and we kept each part only when a measurement supported it. The dominant cost sat in admissions, a capacity-packing problem, so we solved it with an exact model. Our component analysis placed the remaining gap in the nurse layer. We traced it to a solver budget later found still binding on the largest instances (Appendix G), and to a model coupling. On the audited instances no single admission is worth undoing, and a fixed price on where patients go gains no more than its placebo. The coupling loop is the next step.

more time. The single seed shows one deterministic path; Appendix B reports the seed spread.

⁸A certified optimality gap is the distance between our objective and a proven lower bound on the true optimum, as a percentage of that bound (Wolsey, 1998). The true optimum cannot sit below the bound, so the real gap to it is at most this figure (Appendix D).

References

- Barr, R. S., Golden, B. L., Kelly, J. P., Resende, M. G. C., & Stewart, W. R. (1995). Designing and reporting on computational experiments with heuristic methods. *Journal of Heuristics*, 1(1), 9-32.
- Burke, E. K., De Causmaecker, P., Vanden Berghe, G., & Van Landeghem, H. (2004). The state of the art of nurse rostering. *Journal of Scheduling*, 7(6), 441-499.
- Cardoen, B., Demeulemeester, E., & Beliën, J. (2010). Operating room planning and scheduling: A literature review. *European Journal of Operational Research*, 201(3), 921-932.
- Ceschia, S., Rosati, R. M., Schaerf, A., Smet, P., Vanden Berghe, G., & Zanazzo, E. (2025a). *The Integrated Healthcare Timetabling Competition 2024: Problem description and rules*. Competition problem specification, last updated 10 January 2025. <https://ihtc2024.github.io>
- Ceschia, S., Rosati, R. M., Schaerf, A., Smet, P., Vanden Berghe, G., & Zanazzo, E. (2025b). The Integrated Healthcare Timetabling competition 2024. *Operations Research, Data Analytics and Logistics*, 45, Article 200481. <https://doi.org/10.1016/j.ordal.2025.200481>
- Ceschia, S., & Schaerf, A. (2011). Local search and lower bounds for the patient admission scheduling problem. *Computers & Operations Research*, 38(10), 1452-1463.
- Efron, B., & Tibshirani, R. J. (1994). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Fawcett, C., & Hoos, H. H. (2016). Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(4), 431-458.
- Garcia Tercero, L., Van Bulck, D., Devriesere, K., Bakker, S., & Goossens, D. (2026). A multi-neighborhood hybrid approach for the integrated healthcare timetabling competition 2024. *Computers & Operations Research*, 191, Article 107453. <https://doi.org/10.1016/j.cor.2026.107453>
- Glover, F. (1996). Ejection chains, reference structures and alternating path methods for traveling salesman problems. *Discrete Applied Mathematics*, 65(1-3), 223-253.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Hooker, J. N. (1995). Testing heuristics: We have it all wrong. *Journal of Heuristics*, 1(1), 33-42.
- Hoos, H. H., & Stützle, T. (2004). *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann.
- Integrated Healthcare Timetabling Competition (IHTC). (2024). *IHTC-2024: Instances, official validator, and published team results*. <https://ihtc2024.github.io>
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680.
- Maniezzo, V., Stützle, T., & Voß, S. (Eds.). (2010). *Matheuristics: Hybridizing metaheuristics and mathematical programming* (Annals of Information Systems, Vol. 10). Springer.
- Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11), 2600-2606.
- Pisinger, D., & Ropke, S. (2010). Large neighborhood search. In M. Gendreau & J.-Y. Potvin (Eds.), *Handbook of metaheuristics* (2nd ed., pp. 399-419). Springer.

- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455-472.
- Schrimpf, G., Schneider, J., Stamm-Wilbrandt, H., & Dueck, G. (2000). Record breaking optimization results using the ruin and recreate principle. *Journal of Computational Physics*, 159(2), 139-171.
- Shadish, W. R., Cook, T. D., & Campbell, D. T. (2002). *Experimental and Quasi-Experimental Designs for Generalized Causal Inference*. Houghton Mifflin.
- Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming (CP 1998)* (Lecture Notes in Computer Science, Vol. 1520, pp. 417-431). Springer.
- Spearman, C. (1904). The proof and measurement of association between two things. *American Journal of Psychology*, 15(1), 72-101.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80-83.
- Wolsey, L. A. (1998). *Integer programming*. Wiley-Interscience.

A The operating-theatre and nurse models

The report gives the admission model (PAS-MIP) and describes the last two exact models in words. The formulations below are our own and match our implementation in Python. They follow the standard assignment-model forms for operating-theatre scheduling (Cardoen et al., 2010) and for the nurse-to-room subproblem as the competition defines it (Ceschia et al., 2025b). We adapted both to this competition’s costs and constraints.

OT-MIP, per day. For a fixed day d , the surgeries are the patients admitted on day d , and t ranges over the theatres available on day d ($A_{t,d} > 0$). Variables: $y_{p,t}$ (surgery p in theatre t), ω_t (theatre t used), $h_{u,t}$ (surgeon u uses theatre t), and $\tau_u \geq 0$ (the number of extra theatres of surgeon u).

$$\begin{aligned}
\min \quad & W_5 \sum_t \omega_t + W_6 \sum_u \tau_u \\
\text{s.t.} \quad & \sum_t y_{p,t} = 1 \quad (\text{each surgery } p) & (\text{one theatre}) \\
& \sum_p \delta_p y_{p,t} \leq A_{t,d} & (\text{H4}) \\
& \omega_t \geq y_{p,t}, \quad h_{u,t} \geq y_{p,t} & (\text{open / surgeon flags}) \\
& \tau_u \geq \sum_t h_{u,t} - 1 & (\text{S6 transfers}) \\
& y_{p,t}, \omega_t, h_{u,t} \in \{0, 1\}, \tau_u \geq 0 & (\text{domains})
\end{aligned}$$

NRA-MIP. With the layout fixed, (r, s) ranges over occupied room-shifts. Variables: $v_{n,r,s}$ (nurse n , on duty in shift s , covers room r), $o_{n,s} \geq 0$ (workload of n in shift s above the cap), and $c_{e,n}$ (entity e , a patient or occupant, is seen by nurse n). The skill shortfall $\text{gap}_{r,s,n} = \sum_{e \in (r,s)} \max(0, \text{req}_{e,s} - \sigma_n)$ is a constant. The workload demanded in room r at shift s is $w_{r,s} = \sum_{e \in (r,s)} w_{e,s}$. The input gives occupants the same per-shift workload and skill-requirement data as patients. The entities $e \in \mathcal{P} \cup \mathcal{O}$ are the admitted patients and occupants. Each entity e occupies room r_e , and $e \in (r, s)$ means e is present in room r during shift s . Requirements are indexed from each entity’s start. For a patient admitted on day d_0 , stay-shift j is global shift $s = 3d_0 + j$. Occupants are indexed from day 0.

$$\begin{aligned}
\min \quad & W_2 \sum_{r,s,n} \text{gap}_{r,s,n} v_{n,r,s} + W_3 \sum_{e,n} c_{e,n} + W_4 \sum_{n,s} o_{n,s} \\
\text{s.t.} \quad & \sum_{n \in \mathcal{N}_s} v_{n,r,s} = 1 \quad (\text{each occupied } (r, s)) & (\text{H8}) \\
& o_{n,s} \geq \sum_r w_{r,s} v_{n,r,s} - L_{n,s} & (\text{S4}) \\
& c_{e,n} \geq v_{n,r_e,s} \quad (\text{each shift } s \text{ of } e\text{'s stay}) & (\text{S3}) \\
& v_{n,r,s}, c_{e,n} \in \{0, 1\}, o_{n,s} \geq 0 & (\text{domains})
\end{aligned}$$

The greedy nurse assignment warm-starts the NRA-MIP, and the current theatre placement warm-starts the OT-MIP. Each solve therefore begins from a feasible incumbent, which gives branch and bound an upper bound to prune against from the first node (Wolsey, 1998).

In both models every auxiliary counter ($\omega_t, h_{u,t}, \tau_u, o_{n,s}, c_{e,n}$) is only bounded from below. Minimization then drives each to its exact count at optimality. This is the standard linearization device (Wolsey, 1998).

B Seed distribution (five fixed seeds)

This table is the seed-robustness evidence behind the best-of-five headline. A randomized method gives a different result on each run, so a single run cannot describe it. The reporting standard for heuristics is several runs with their spread, not one number (Barr et al., 1995). For each instance the table gives the number of feasible seeds, the best, mean, worst, and standard deviation of the objective, and the best-of-5 gap to best-known. Feasibility never depends on the seed. The spread of the objective shows how much of the headline comes from restarts. Each seeded run is deterministic, so this table reproduces on the same solver versions. Per-seed feasibility is checked

by our golden-tested evaluator. The official validator re-checks each instance’s reported best, since that is the only per-seed value we report. The full per-seed objective and runtime for every instance are in `results/seed_runs.csv` in our GitHub repo.

inst	#seeds	best	mean	worst	std	best gap%
i01	5	4047	4084	4105	27	+5.3
i02	5	1444	1460	1474	12	+14.2
i03	5	10605	10627	10640	15	+1.1
i04	5	2021	2092	2181	59	+7.3
i05	5	12903	12936	13005	40	+1.1
i06	5	10696	10698	10700	2	+0.2
i07	5	5530	5697	5882	130	+10.0
i08	5	6929	6960	7034	43	+10.1
i09	5	7150	7163	7188	16	+7.0
i10	5	21525	21545	21550	11	+3.4
i11	5	25966	25984	26020	23	+0.1
i12	5	13453	13551	13763	122	+8.2
i13	5	17979	18014	18029	20	+3.8
i14	5	10011	10098	10157	55	+2.7
i15	5	13451	13789	14158	320	+7.7
i16	5	11485	11528	11575	34	+13.3
i17	5	47970	49596	51240	1288	+18.3
i18	5	37903	38041	38246	164	+0.6
i19	5	49544	49859	50055	260	+11.1
i20	5	30098	30110	30123	9	+3.4
i21	5	28011	28434	28768	287	+13.4
i22	5	53380	53546	53701	122	+11.5
i23	5	39230	39334	39405	81	+4.5
i24	5	34476	34539	34566	37	+3.8
i25	5	12322	12710	13398	428	+7.0
i26	5	71048	71343	72064	415	+10.0
i27	5	62222	63009	63963	856	+20.1
i28	5	76176	76204	76230	25	+1.3
i29	5	14287	14369	14535	96	+14.5
i30	5	40597	40743	40895	109	+7.0

Best-of-5: aggregate gap 7.8%. Mean per-instance gap 7.4% (best-of-5) vs 8.4% (average single seed); restarts recover 1.0 points.

C Component-level cost breakdown

A total-versus-total comparison says who is ahead but not why, so we split the gap by cost term. This follows the case for experiments that explain where a method’s performance comes from, rather than ranking methods alone (Hooker, 1995). The table gives each soft term’s weighted cost, summed over the 30 instances, for our solutions and for the best-known solutions, with the difference and its share of the total gap. Our costs are read from the official validator logs, and the best-known costs come from running the same validator on the published reference solutions (IHTC, 2024). The nurse terms S2, S3, S4 make up about 90% of the net gap (78% of the gross positive differences). We sit below best-known on the weighted S8. That cost mixes per-instance weights, so Table 3 counts admitted optionals directly (2995 versus 2933, 62 more). The script `report/make_admissions.py` counts them from our solution files in `results/` and from the same reference solutions.

soft term	our cost	best-known	difference	share of gap%
S1 age mix	7068	3251	+3817	+6.8
S2 nurse skill	38704	22197	+16507	+29.5
S3 continuity	123096	101331	+21765	+38.9
S4 workload	22004	9683	+12321	+22.0
S5 open OTs	33340	32680	+660	+1.2
S6 surgeon transfer	842	1318	-476	-0.9
S7 delay	137655	128150	+9505	+17.0
S8 unscheduled	409750	417950	-8200	-14.7
total	772459	716560	+55899	100

Table 3: Optional patients admitted per instance (opt = optional patients; ours vs the reference solutions), counted directly rather than from the weighted S8.

inst	opt	ours	ref	inst	opt	ours	ref	inst	opt	ours	ref
i01	28	20	20	i11	84	35	35	i21	257	234	232
i02	25	25	25	i12	47	18	18	i22	210	141	147
i03	38	19	19	i13	102	89	91	i23	269	172	161
i04	18	18	18	i14	59	48	48	i24	137	55	56
i05	87	53	53	i15	102	95	95	i25	211	195	194
i06	77	11	11	i16	29	17	21	i26	281	211	213
i07	54	54	54	i17	303	270	275	i27	370	322	322
i08	36	36	34	i18	196	85	85	i28	191	49	49
i09	89	89	89	i19	292	225	177	i29	208	190	193
i10	107	64	65	i20	80	36	39	i30	196	119	94
Total 2995 of 4183 optionals admitted, versus 2933 in the reference solutions (+62).											

D Certified lower bounds

We bound the true optimum from below by relaxing the exact model. A dropped constraint can only lower the optimum, so a relaxation’s optimum is a valid lower bound (Wolsey, 1998). Ceschia and Schaefer (2011) bound patient admission scheduling the same way. Our relaxation is the admission model of Section 2 with the room dropped (binary $x_{p,d}$, admit p on day d). The large-instance clipping of optional windows noted in Section 2 is not applied here. A valid relaxation needs every legal admission day, and `ihp/bounds.py` keeps them all. Over the whole horizon this model is paired with the per-day theatre variables of Appendix A. Those variables are the binary theatre choice plus the open-theatre and surgeon-transfer indicators, all under the per-theatre H4. The relaxation keeps the admission rules H5 and H6 and the surgeon-day H3 and theatre-day H4 time limits. It also keeps the four costs those resources drive, S5, S6, S7 and S8. Those four are all it minimizes. It drops the room and gender assignment (H1, H2, H7), the bed capacity they imply, and the nurse coverage (H8). The same drops remove the non-negative costs S1 (age mix) and S2 to S4 (nurse). The removals enlarge the feasible region, and every cost they drop is non-negative. Neither change can raise the optimum, so the bound is valid.

Each relaxation runs on Gurobi with a deterministic `WorkLimit = 120`, `Threads = 1`, `Seed = 0` and Gurobi’s default MIP gap, with no wall-clock limit. The reported bound is `[ObjBound]`, the proven dual bound. Branch and bound keeps a proven floor under every subproblem it has not yet closed, so this bound stays valid when the work limit stops the solve early. On i01 to i06 the relaxation solves to proven optimality, so the bound is its exact optimum. On the other 24 (i07 to i30) the bound is the dual bound at the work limit. That bound is still valid, but looser. Per-instance status is in `results/bounds.csv`. The aggregate certified gap is pooled over instances, $g = (\sum_i \text{ours}_i - \sum_i \text{LB}_i) / \sum_i \text{LB}_i$. The table gives each instance’s objective, best-known, bound and both certified gaps. Best-known’s own gap shows how much of ours comes from bound looseness (Section 5).

inst	ours	best-known	LB	cert. gap%	BK cert. gap%
i01	4047	3842	3430	18.0	12.0
i02	1444	1264	655	120.5	93.0
i03	10605	10490	9710	9.2	8.0
i04	2021	1884	865	133.6	117.8
i05	12903	12760	12400	4.1	2.9
i06	10696	10671	10390	2.9	2.7
i07	5530	5026	1659	233.3	203.0
i08	6929	6291	3440	101.4	82.9
i09	7150	6682	2320	208.2	188.0
i10	21525	20820	14320	50.3	45.4
i11	25966	25938	25535	1.7	1.6
i12	13453	12430	6635	102.8	87.3
i13	17979	17328	7355	144.4	135.6
i14	10011	9746	5628	77.9	73.2
i15	13451	12486	6075	121.4	105.5
i16	11485	10139	5615	104.5	80.6
i17	47970	40535	20675	132.0	96.1
i18	37903	37660	36550	3.7	3.0
i19	49544	44587	26145	89.5	70.5
i20	30098	29098	21887	37.5	32.9
i21	28011	24703	10822	158.8	128.3
i22	53380	47861	29061	83.7	64.7
i23	39230	37550	27197	44.2	38.1
i24	34476	33221	30355	13.6	9.4
i25	12322	11517	4605	167.6	150.1
i26	71048	64613	43630	62.8	48.1
i27	62222	51828	29650	109.9	74.8
i28	76176	75172	67215	13.3	11.8
i29	14287	12475	6645	115.0	87.7
i30	40597	37943	21648	87.5	75.3

Aggregate certified gap 57.0% (ours) vs 45.6% (best-known). The bound is valid but loose, since it drops the room, gender and nurse terms. Both gaps are therefore large. The true optimum lies in $[LB, \text{ours}]$.

E The nurse-aware admission audit

The audit is a standalone script (`extensions/postpone_audit.py` in our repository); it consumes the final validated solutions from `results/` and modifies neither the pipeline nor its results. The pipeline works one way only. Every admit-or-postpone decision compares a patient’s own cost against the penalty W_8 , and it never looks at the nurse terms. Once a patient is admitted, no stage re-examines that decision individually against the full objective (the CP-SAT LNS accepts whole rounds, not single admissions, and the descent relocates admitted patients but never removes one). The audit adds the missing move. It works through the admitted optional patients in a seeded random order, so the audit is reproducible. For each such patient p , it removes p and re-scores the complete objective. Changing one thing at a time is how an ablation assigns credit (Fawcett and Hoos, 2016). Before scoring, the nurse assignment is re-solved with the same greedy nurse fill that the pipeline’s true-cost descent uses. The removal is kept only if the total strictly drops; otherwise p is restored exactly. This is first-improvement descent over a one-move neighborhood, with the randomized order and the repeat-until-no-change rule that stochastic local search uses (Hoos and Stützle, 2004). Passes repeat until one finds no improvement, with a cap of four as a safeguard against a long sweep; in every run reported here the first pass already found none, so the cap never bound. A removal of an optional patient cannot violate any hard constraint. The hard constraints fall into three cases. Some bound resource use, and a removal only frees resources. Some apply only to the removed patient’s own placement. The rest concern mandatory patients

(H5). Feasibility is therefore preserved throughout. Any changed solution would be re-solved by the exact OT and NRA models and re-validated by the official validator. The baseline’s theatres and nurses were themselves solved exactly. A changed plan is therefore compared with the baseline like-for-like, not under the sweep’s cheaper greedy roster.

Table 4 reports the outcome on i01 and the five instances with our largest relative gaps to best-known. These were chosen because a volume-driven coupling loss should be largest where the gap is largest, making zero removals a worst-case outcome. We picked the instances where the effect should show most, rather than a convenient sample, so the test carries evidence about the mechanism (Hooker, 1995). No removal is ever accepted. The margin column shows how far each decision is from flipping. For every admitted optional we score $\text{cost}(\text{plan without } p) - \text{cost}(\text{plan with } p)$. Both states are scored under the same greedy nurse fill that the sweep uses. The exact nurse re-solve, far costlier per candidate, is reserved for accepted removals, of which there were none. A positive margin means keeping the patient is cheaper; a negative margin would mark a patient the pipeline should not have admitted. All 1 061 margins are positive, which also makes the zero-removal outcome independent of the sweep order. On these instances the nurse-blind admission rule therefore makes the right call every time, as far as the audit can measure it with the greedy nurse fill. The loss from the admission–nurse link must then lie somewhere other than in how many patients we admit. Two candidates remain. The first is where patients are placed, meaning rooms and days, which is what drives continuity. Section 5 gives that reading. The second is coordinated multi-patient withdrawals, which this one-at-a-time scope cannot test. Appendix F examines them.

Table 4: The postpone audit on i01 and the five largest-gap instances. Margins are in cost units; a negative minimum would mean a net-negative admission. No removal is accepted on any instance.

instance	optionals			margin			net-negative admissions
	W_8	audited	removals	min	median	max	
i01	350	20	0	160	289	334	0
i02	150	25	0	82	110	147	0
i17	500	270	0	240	557	1070	0
i21	250	234	0	70	355	665	0
i27	500	322	0	58	338	595	0
i29	300	190	0	206	280	351	0
total		1 061	0				0

F A nurse-aware upper layer: a pre-registered test with a placebo control

The audit of Appendix E shows that no single admission is worth undoing on the instances it audits. This appendix tests the other possibility, which is where each admitted patient is placed. We add a price for the two nurse costs that can be estimated without solving the nurse model. The design is tested under a protocol written and frozen before the runs, so the results could not shape the criteria; this is the practice of pre-registration (Nosek et al., 2018). Its registration time is recorded in `extensions/nurse_pricing/PROTOCOL.md`. All code, per-run outcomes (`runs.csv`), and the post-hoc analysis script sit alongside it. The runs form a separate result set: the headline results of this paper are unchanged.

Design. Two terms are added to the surrogate objective, which guides the construction, the ALNS, and the moves the descent tries. The exact models are untouched and keep their own objectives (Section 2); every accept-or-reject test still uses the true objective, and nurses are still assigned by the unchanged, exact NRA-MIP. First, a workload term: per shift, $W_4 \cdot \max(0, \text{workload demand} - \text{on-duty capacity})$. This term is a valid lower bound

on S4, since caps fill first and indivisibility only raises the true cost. It is zero whenever the nurses on duty can absorb the demanded workload. Second, a skill-scarcity term: per day, $W_2 \cdot \max(0, \text{rooms whose daily requirement reaches the top skill level} - \text{top-skill nurses on duty})$. The day’s supply is the smallest of its three shifts. We assume one top-skill room per top-skill nurse is manageable. Patient requirements enter per day of stay, since requirements fall as patients recover (the max over a whole stay is degenerate on these instances: on i01 every patient’s maximum is the same level). Continuity (S3) is deliberately not priced: at design time, Appendix E and Section 5 read it as a property of the roster, and no fixed price works without passing information between the layers. The budget probe of Appendix G later showed that a larger nurse solve also recovers part of the continuity cost. A control run with both terms off reproduces the recorded baseline exactly, verified bit for bit on i01, seed 1. The added code itself therefore changes nothing, and every same-seed difference comes from the priced terms alone.

Protocol. Five instances influenced the design (i01, i02, i17, i21, i29) and are declared a development set, reported without claims. An earlier term that priced the spread of skill levels within each room-day failed on them, and the per-day skill-level check above used them. Instance i27, audited in Appendix E, contributed no design decision to the pricing terms and therefore stays held out. The remaining *25 instances are held out*: no design decision ever touched them, and all confirmatory claims rest on them alone. This is the standard split between data that shapes a method and data that judges it (Hastie et al., 2009). The weights of both terms were frozen at 1.0 with no tuning, so no result could feed back into the design (Nosek et al., 2018). All 30 instances were run on all five seeds under the production budgets, and all comparisons are best-of-five against best-of-five over the same seeds, with gaps total-based over the frozen best-known snapshot. The seeds are held fixed across both arms, so every run is paired with its own baseline and seed variation cannot be read as an effect of the change (Barr et al., 1995). Every objective reported in this appendix is the official validator’s, with zero violations across all 300 runs of both arms. We declared two success criteria in advance. Criterion (a) is the aggregate effect. The held-out best-of-5 aggregate gap must improve. Criterion (b) tests whether the effect is broad and not concentrated in a few large wins. A majority of the 125 held-out same-seed pairs must not worsen.

Outcome. Both criteria were met: the held-out aggregate gap moved from 6.78% to 6.39% (−0.39 points), and 56 improved and 7 unchanged against 62 worse. This is the narrowest possible majority, and among strictly decided pairs more got worse. Post-hoc analysis (labeled as such: it was not pre-specified) shows the aggregate improvement is real but fragile, with a 95% bootstrap interval over instances of [−0.81, −0.03] points (20,000 percentile resamples). The per-instance effect, though, is indistinguishable from zero (13 better, 12 worse; two-sided Wilcoxon signed-rank on relative per-instance changes, $n=25$, $p = 0.68$).⁹ The gain is carried by a few large wins over the 25 held-out instances (sum −3 035) against many small losses (sum +548). The pre-declared concentration prediction (larger gains where nurse cost dominates) is not supported by rank correlation (Spearman, 1904) (Spearman of relative change against nurse-cost share, $n=25$, +0.20, the wrong direction).

Placebo control. The decisive question is causal: does the improvement come from nurse-supply information, or would any comparable change to the numbers guiding the search do the same? A control that removes only the claimed ingredient answers this (Shadish et al., 2002). The placebo

⁹A bootstrap interval estimates how much a figure would vary if the instance set itself were drawn again (Efron and Tibshirani, 1994). `analyze_posthoc.py` redraws the 25 held-out instances with replacement and recomputes the aggregate-gap change. It repeats this 20,000 times and reports the middle 95% of the results. The Wilcoxon test (Wilcoxon, 1945) ranks the paired changes by absolute size and asks whether the signs lean one way. It makes no assumption about the shape of the distribution, so a few large wins cannot decide the result on their own. That suits these 25 changes. Its p is the probability of a lean at least this strong if there were no true per-instance effect.

has a second pre-registered protocol (`PROTOCOL_PLACEBO.md`). We re-ran the full campaign with the identical terms priced against a shuffled roster. Each shift keeps a capacity and each day keeps a top-skill count, but drawn from the wrong shift or day. The sizes therefore stay identical, and only the timing is false. The placebo reproduces the gain within noise (-0.40 points against the nurse-informed arm’s -0.39 , difference $+0.006$ points with a bootstrap interval of $[-0.19, +0.19]$). By the declared decision rule, *the cause is not established*: aligning the price with the true roster adds no advantage this experiment can measure, since the interval spans half the total effect. The protocol declares one limit of this control. It removes the alignment with the real roster, not the information itself. Both terms still read the true patient demand; only the supply numbers are shuffled. The follow-up control below would remove the demand information too.

What both arms actually did. A decomposition of the improvements by cost component explains the tie. The real patch does reduce its priced skill term about twice as much as the placebo ($-1\,924$ versus -929), a trace of the intended mechanism. But the placebo reduces workload twice as much, and both arms share one dominant pattern, consistent with the added term working like a *tax on admissions*. Over the 25 held-out instances, both arms leave more optional patients out (about two dozen, almost all on i19) at total unscheduled penalty $+4\,900$ and $+5\,400$. They gain back more than that through lower delay and nurse cost ($-7\,470$ and $-7\,765$; the remaining components account for the small residual). This is consistent with Appendix E: removing any *single* admitted optional from a fixed plan never pays, yet a tax-induced withdrawal of several patients, re-planned around their absence, does. This is the coordinated withdrawal that the audit’s patient-by-patient scope could not test (i19 itself lay outside the audited set, so the audit constrains neither reading there). The large W_8 pushes the search to admit as many patients as it can, and it seems to go a little too far. Only coordinated changes can undo that.

Conclusion of the test. A fixed price on where patients are placed, built from the supply side we tested, improves the search no more than an equal-sized change carrying no real nurse information. We have now measured both routes, and neither fixed nurse-informed price has beaten its own placebo. This strengthens the conclusion of Section 5: closing the coupling’s share of the nurse gap requires the admission and nurse layers to exchange information, not a fixed surcharge. Appendix G separates that share from the solver-budget share. The clean follow-up control carries no nurse information at all. The protocol names a random room-day price of matched size. For the tax reading, the most direct version is a flat per-admission surcharge. If either reproduces the -0.4 -point shift, the effect is entirely the tax.

G Where the nurse gap sits: a structure comparison and a budget probe

The component analysis of Section 5 places most of the remaining gap in the three nurse terms. This appendix measures where that gap sits, in two steps, using only stored solutions (`extensions/structure_diff/`, with the probe protocol, scripts, and per-instance records alongside). The structure metrics come from our golden-tested evaluator, and the probe scores come from the official validator. The runs form a separate result set; the headline numbers are unchanged.

Structure comparison. We compared the structure of our public-set solutions with the competition’s published best solutions on the same instances (IHTC, 2024). Each metric below tests one possible reason for the nurse gap, so the comparison can tell the reasons apart instead of only showing that a gap exists (Hooker, 1995). The published best solutions do not win by lining up patients’ stays or by using fewer rooms. Mean roommate stay overlap is 0.13 on both sides,¹⁰

¹⁰The overlap of a roommate pair is the number of days both patients are in the room, divided by the number of days at least one of them is there (`compare_structure.py`). Identical stays give 1.0 and disjoint stays give 0 .

and patients per used room match (9.8 ours, 9.7 theirs). Their advantage sits in all three nurse terms, and their mean count of distinct nurses per patient is 4.95 against our 5.56. The layouts are similar, so this pointed at the nurse solve rather than the layout.

Budget probe. That reading was registered in advance in `PROBE.md`, again so the outcome could not shape the test (Nosek et al., 2018); the file also records two earlier exploratory runs. We then re-solved the NRA-MIP once on each stored layout at work budget 3000, several times the production ceiling of 600 to 800, so a budget shortage could not be what limits the result. The layout stays untouched. Only the budget changes, so any improvement is attributable to it alone. This is again the one-change logic of an ablation (Fawcett and Hoos, 2016). A probe built this way answers why the gap is there. Competitive comparison against other methods cannot answer that (Hooker, 1995). Table 5 shows the outcome. On the six largest-gap nurse-heavy instances the re-solve closes between 6% and 63% of the gap to best-known, mostly in continuity. The pooled share over the seven probed instances is 21.9%, and the small control instance closes nothing, as the protocol predicted.

Table 5: The budget probe: one NRA-MIP re-solve at work budget 3000 on the frozen stored layout, scored by the official validator. The share is the fraction of the instance’s gap to best-known that the re-solve closes. The i17 and i22 runs were exploratory, as the protocol records.

instance	before	after	change	share of gap closed
i21	28 011	25 926	−2 085	63.0%
i17	47 970	45 030	−2 940	39.5%
i29	14 287	13 836	−451	24.9%
i22	53 380	52 715	−665	12.0%
i19	49 544	48 959	−585	11.8%
i27	62 222	61 572	−650	6.3%
i02 (control)	1 444	1 444	0	0.0%

What this changes. The nurse gap splits into a solver-budget share and a residual share. The budget share is solver time that the production budgets left unused on the largest instances. The residual share is what remains for the admission–nurse coupling. Appendix E checks one admission at a time and finds none worth undoing. Appendix F finds a fixed price no better than a placebo. The diagnosis of Section 5 survives this correction, at a smaller measured scale. The case for the coupling loop as future work now stands next to a simpler first step, a larger or restructured nurse solve. One further test could sharpen the split: running our nurse model, at a large budget, on the published best layouts. If it matched their nurse costs there, the residual would point at the layout; if it fell short, at our nurse solve. We note it as a possible next step.

H Held-out evaluation on the hidden instances

Our development and tuning used the 30 public instances only, and every result so far was measured on them. The competition kept a second set of 30 hidden instances for its final evaluation and has since published both the instances and the best solutions its finalists reached. A single campaign of the frozen pipeline on that set measures generalization directly, the standard role of a held-out set (Hastie et al., 2009). The protocol (`extensions/hidden_eval/PROTOCOL_HIDDEN.md`) was registered before any run, so no result could shape it (Nosek et al., 2018). It fixed the code, the budgets, and the five-seed best-of-five policy. It also declared that we would report the outcome, whatever it was, and give the feasibility and runtime of every run (Barr et al., 1995). The benchmarks are the published best solutions, re-scored by the official validator; the values match the UB column of the competition paper (Ceschia et al., 2025b) on every instance we spot-checked.

Outcome. All 150 runs are feasible under the official validator. The total-based aggregate gap to the published best solutions is 6.10%, against 7.80% on the public set. Thirteen of the 30 instances fall within 3% and 18 within 5%, and the largest gaps are again on nurse-heavy instances (17.8% on m27). The set includes instances larger than any public one (m29 has 631 patients). Table 6 gives the per-instance outcomes.

Two caveats. The hidden benchmarks come from the five finalist methods, each run ten times by the organizers (Ceschia et al., 2025b), while the public best-known values pool the submissions of 32 teams. The hidden benchmarks may therefore be easier to approach, and the two aggregate gaps are not strictly comparable. One campaign also cannot show how much the result would vary if we ran it again; the per-seed records are in `extensions/hidden_eval/`. With those limits stated, the method’s performance carries to instances it never saw, at a similar gap level, with feasibility on every run.

Table 6: The held-out evaluation: the frozen pipeline, run once on the competition’s hidden instances under the pre-registered protocol of `extensions/hidden_eval`, best of five seeds per instance, scored by the official validator. The benchmark is the best published solution per instance, re-scored by the same validator. Runtime is the solver time summed over the five seeds.

instance	best-of-5	published best	gap%	runtime (s)
m01	3 505	3 384	3.58	1 836
m02	12 246	12 211	0.29	1 937
m03	6 777	6 697	1.19	1 323
m04	3 646	3 345	9.00	2 968
m05	12 054	11 956	0.82	1 496
m06	28 635	28 250	1.36	8 049
m07	7 388	7 329	0.81	8 800
m08	15 348	14 976	2.48	7 909
m09	33 481	32 967	1.56	5 294
m10	26 404	26 027	1.45	6 349
m11	36 214	35 030	3.38	8 820
m12	13 097	11 681	12.12	9 305
m13	31 656	31 189	1.50	9 689
m14	13 383	13 368	0.11	7 975
m15	32 828	28 727	14.28	10 019
m16	18 943	17 271	9.68	9 059
m17	23 316	22 542	3.43	9 828
m18	8 679	7 817	11.03	10 017
m19	23 262	21 790	6.76	9 636
m20	6 112	5 335	14.56	11 165
m21	25 783	25 170	2.44	10 046
m22	24 748	21 330	16.02	10 003
m23	19 225	18 301	5.05	9 330
m24	31 788	30 873	2.96	10 132
m25	34 741	33 985	2.22	6 441
m26	72 379	69 730	3.80	9 861
m27	33 005	28 028	17.76	10 746
m28	53 640	45 913	16.83	12 294
m29	49 570	48 082	3.09	18 927
m30	35 472	31 649	12.08	11 218